

A-RMS · 원인 분석 보고서

wiki 동시 편집 동기화 불일치 원인 분석 및 개선안

분석 대상	Java-Service-Tree-Framework-Frontend-Web Java-Service-Tree-Framework-Broker-Hub
증상	① 같은 내용이 복불처럼 중복 삽입 ② 편집 내용 유실·의도치 않은 내용 반영
분석 방법	정적 코드 분석 (브라우저 실행·재현 검증 미수행)
작성일	2026-08-07
산출 경로	tasks/wiki-concurrent-edit-sync/

본 문서의 각 항목은 **사실**(코드로 확인)과 **가설**(추론)을 구분해 표기한다. 가설 항목은 실측 검증 전까지 확정된 원인으로 취급하지 않는다.

wiki 동시 편집 동기화 — 원인 분석

요약

동기화 채널은 블록 단위 LCS diff 하나뿐이고, 서버는 그 diff를 검증·정렬·변환 없이 그대로 되뿌리는 릴레이다. 서버에 OT 구현(`OtService` / `OtController`, revision 관리)이 존재하지만 위키 에디터 클라이언트는 그 경로를 전혀 쓰지 않는다 (프론트가 publish 하는 destination은 `/app/join`, `/app/leave`, `/app/selection` 3개뿐 — `session-manager.js:135,186,208`).

핵심 결함 2개:

- `lastBlocks`가 원격 반영 시 갱신되지 않는다 (`adms.js:235` vs `164`). 이것 하나로 중복 삽입이 **echo** 없이도 결정론적으로 재현된다. → 원인 A의 주범.
- 블록 통째(`outerHTML`) **replace** 단위라서 같은 블록을 두 사람이 만지면 나중 것이 앞의 것을 통째로 덮는다. 게다가 base가 어긋난 상태에서 `applyBlockDiff`의 방어 가드가 **delete/replace**만 조용히 삼키고 **insert**는 그대로 실행한다(`collab-diff.js:123,139` vs `131`). → 원인 B.

선행 가설 4건 중 1번은 반증(서버 echo는 있으나 `session-manager.js:165`에서 이미 필터됨), 2·4번은 사실 확인, 3번은 단독 원인으로는 반증(증폭 요인).

동기화 구조

송신 (`adms.js:157-168`) `keyup` → 150ms debounce → `getEditorBlocks` (body 직계 자식의 `outerHTML` 배열, `data-remote-cursor` 제외) → `computeBlockDiff(lastBlocks, current)` → `lastBlocks = current` 선반영 → `sessionManager.remoteUserUpdate(JSON.stringify({type:"diff", ops}))`

서버 (`EditorController.java:152-200`) `/app/selection` → Redis에 상태 저장 → `messagingTemplate.convertAndSend("/topic/sessions/{sid}/selections/document/{did}", message)` — 원본 메시지 그대로, 발신자 포함 전 구독자에게 **broadcast**. 순번·revision·변환 없음.

수신 (`session-manager.js:162-171` → `adms.js:229-250`) `response.userInfo.id !== self._userInfo.userId`로 자기 것 필터 → `onContentsChange` → `type==="diff"`면 `applyBlockDiff(editor, parsed.ops)`

세션 키: `sessionId = UUID.nameUUIDFromBytes(documentId)` (`SessionController.java:59`) — 결정적이라 같은 위키를 연 사람끼리 같은 토픽에 모인다.

원인 A: 중복 삽입

A-1. `lastBlocks` stale — 주원인

근거(파일:줄) - `adms.js:139` `var lastBlocks = []` (`contentDom` 바깥 스코프) - `adms.js:164` 송신 시에만 `lastBlocks = currentBlocks.slice()` - `adms.js:235` `applyBlockDiff(...)` — `lastBlocks` 갱신 없음.
`onContentsChange` 는 `lastBlocks` 와 같은 스코프(`adms.js:229`)라 갱신 가능한데 하지 않는다 - `collab-diff.js:109-150` `applyBlockDiff` 는 DOM만 직접 조작 → `dataReady` 미발생 → `adms.js:145-149` 의 재동기화 방어가 작동하지 않음 - `collab-diff.js:131-137` `insert` 는 `children[pos]` 존재 여부와 무관하게 항상 실행

사실|가설: 사실 (코드 확인)

재현 시나리오 (A·B 모두 `[P1,P2]` , `lastBlocks` 모두 `[P1,P2]`)

#	이벤트
1	B가 끝에 새 문단 P3 입력 → B DOM <code>[P1,P2,P3]</code>
2	B keyup+150ms → <code>ops=[retain,retain,insert P3]</code> , <code>lastBlocks_B=[P1,P2,P3]</code> , publish
3	A 수신 → <code>applyBlockDiff</code> → A DOM <code>[P1,P2,P3]</code> , 그러나 <code>lastBlocks_A</code> 는 여전히 <code>[P1,P2]</code>
4	A가 P1을 수정 → A DOM <code>[P1',P2,P3]</code>
5	A keyup+150ms → <code>computeBlockDiff([P1,P2], [P1',P2,P3]) = [replace P1', retain, **insert P3**]</code>
6	B 수신 → <code>children=[P1,P2,P3]</code> . <code>replace(0)</code> OK, <code>retain(1)</code> , <code>insert</code> 시 <code>children[2]=P3</code> 가 존재하므로 <code>insertBefore(P3사본, P3)</code>
7	B DOM = <code>[P1',P2,P3사본,P3]</code> — P3 중복

7 이후 A(3블록)와 B(4블록)의 길이가 어긋나 이후 모든 op의 인덱스가 밀린다. echo 여부와 무관하게 성립한다.

A-2. 중복이 "쌍이기만" 하는 비대칭

근거: `collab-diff.js:123` (`delete` 는 `children[pos]` 가 없으면 skip), `:139` (`replace` 도 동일), 반면 `:131-136` (`insert` 는 `children[pos]` 가 없으면 `appendChild` 로 무조건 성공)

사실|가설: 사실

base가 어긋나면 삭제·치환은 조용히 무시되고 삽입만 살아남는다. 그래서 증상이 "가끔 어긋남"이 아니라 "중복만 단조 증가"로 나타난다.

A-3. 자기 diff 재적용 (선행 가설 1) — 반증

근거 - 서버는 발신자 포함 broadcast가 맞다 (`EditorController.java:191` `convertAndSend` , 발신자 제외 로직 없음) - 그러나 `session-manager.js:165` `if (response.userInfo.id !== self._userInfo.userId)` 에서

diff/cursor 구분 없이 이미 걸러진다. 양쪽 모두 String (`common.js:834` `userID = json.sub` , `UserInfo.java:6` `private String id`) → 타입 불일치 없음

사실|가설: 사실 (반증)

`adms.js:240` 의 `String(contents.id) !== String(userInfo.userId)` 는 이 필터가 있는 한 도달 불가능한 이중 방어다. 과거에 이 문제를 의심했던 흔적으로 보이나, 현재 중복 삽입의 원인은 아니다.

A-4. 동일 블록 LCS 모호성 (선행 가설 3) — 단독 원인으로서는 반증, 증폭 요인

근거: `collab-diff.js:12` `outerHTML` 문자열, `:32,:40` `===` 비교, `:44` 동점 시 insert 우선

사실|가설: 비교 방식은 사실, 영향 범위는 가설

`applyBlockDiff` 가 `computeBlockDiff` 의 인덱스 규약을 정확히 미러링하므로, 수신자 상태 == 송신자 `lastBlocks` 인 한 LCS가 어느 쪽을 매칭하든 결과는 수렴한다(시각적으로 엉뚱한 블록이 지워질 뿐). 따라서 이것만으로 발산하지는 않는다. 다만 빈 `<p><br></p>` 가 여러 개인 실제 문서에서는 A-1로 base가 어긋난 순간 오정렬 확률과 파괴력을 크게 키운다.

원인 B: 내용 유실·오반영

B-1. 블록 통째 replace → 같은 문단 동시 편집 시 스왑·소실

근거: `collab-diff.js:57` (`delete+insert` → `replace`), `:138-148` `body.replaceChild(newElRep, children[pos])` , `adms.js:170` 150ms debounce

사실|가설: 사실

블록 내부 텍스트를 병합하는 코드가 없다. `replaceChild` 는 로컬 캐럿이 놓인 노드도 통째로 교체한다.

재현 시나리오 ①(소실) — 두 사람이 같은 문단 P1 편집

1. A가 P1에 타이핑(디바운스 대기 중, 아직 미전송)
2. B의 타이머가 먼저 만료 → `replace P1b` publish
3. A 수신 → `replaceChild(P1b, P1)` → A가 방금 친 글자와 캐럿이 소실
4. A 타이머 만료 → `computeBlockDiff([P1,P2], [P1b,P2])` → B가 이미 가진 P1b를 되돌려 보냄 → A의 입력은 어디에도 남지 않음

재현 시나리오 ②(오반영·핑퐁) — 양쪽이 거의 동시에 전송

1. A publish `replace P1a` (base `[P1,P2]`), B publish `replace P1b` (같은 base) — 변환 없이 교차
2. A 수신 → A 화면 = **P1b**. B 수신 → B 화면 = **P1a**. `lastBlocks_A=[P1a,..]` , `lastBlocks_B=[P1b,..]`
3. A가 이어서 타이핑 → base `P1a` 기준 diff → `replace P1b+수정` 전송 → B 화면이 뒤집힘. 반대도 동일
4. "내가 쓴 게 상대 화면에, 상대 게 내 화면에" 상태가 계속 순환

B-2. 전송 실패 시 영구 유실 (재전송 없음)

근거 - `adms.js:164` 가 `remoteUserUpdate` (:165) 호출 전에 `lastBlocks` 를 전진시킨다 → 전송 실패해도 그 변경분은 다시는 diff에 나오지 않음 - `session-manager.js:192-194` `if (!this._sessionId || !this._documentId) return;` — 조용히 폐기 - `session-manager.js:61-64` `_handleConnectionFailure` → `this._pendingActions = []` — 큐에 쌓인 미전송 diff 전부 폐기 - `session-manager.js:208` `publish` — ack·재시도·시퀀스 없음

사실|가설: 사실

재현 시나리오: 네트워크 순단 → A가 5초간 계속 타이핑 → 각 `sendDiffIfChanged` 가 `lastBlocks` 전진 후 `_ensureConnected` 로 큐잉 → `onWebSocketError` 발생 → `_pendingActions=[]` → 5초분 편집이 아무 로그·경고 없이 소멸, 이후 diff는 새 base 기준이라 복구 불가.

B-3. 브로드캐스트 순서 보장 없음 (선행 가설 4 확장)

근거 - ops payload에 base version/seq 없음 (`adms.js:165`) - `WebSocketConfig.java:17-20` — `config.enableSimpleBroker("/topic")` 만 호출. `setPreservePublishOrder(true)` 미설정(기본 false), `configureClientInboundChannel` 미오버라이드(기본 스레드 풀)

사실|가설: 설정 부재는 사실, 실제 역전 발생은 가설(부하 의존)

Spring STOMP는 기본값에서 동일 세션의 연속 메시지를 서로 다른 스레드로 처리·전달할 수 있다. 순서가 뒤집힌 두 diff는 인덱스 기반이라 전혀 다른 문서를 만든다. 150ms debounce는 빠른 입력에서 연속 전송을 만들어 이 구간에 정확히 걸린다.

B-4. sanitize/ firstChild 에 의한 조용한 드롭

근거: `collab-diff.js:79` (`script/iframe/object/embed` 제거) vs `editor-operation.js:11` `allowedContent: true`, `:10` `extraPlugins: ["drawio", ...]` / `collab-diff.js:130,142` `tmp.firstChild` — null이면 아무것도 하지 않고 통과

사실|가설: 비대칭 자체는 사실, 실제 발생 빈도는 가설(drawio/accordion 위젯이 body 직계에 `iframe` / `object` 를 만드는지에 의존)

sanitize 후 남는 게 없으면 `insert` 는 무시되고 `replace` 는 원본을 그대로 둔다. 송신자 화면에는 있고 수신자 화면에는 없는 블록이 생기며, 이후 diff는 이 사실을 모른 채 인덱스를 계산한다.

B-5. 원격 커서 span이 저장 본문에 섞임

근거: `collab-diff.js:238` `editorDoc.body.appendChild(cursor)` — 커서를 에디터 body 안에 삽입. `adms.js:283-292` 저장 직전 처리는 `.panel-body` 비우기뿐, 커서 제거 없음. `adms.js:303` `getData()`. `editor-operation.js:11` `allowedContent: true` → CKEditor 출력 필터가 걸어내지 않음

사실|가설: 가설 (경로는 사실, `getData()` 출력에 실제 포함되는지는 미실행 검증)

성립하면 저장본에 상대 사용자 이름 라벨과 인라인 style이 그대로 박힌다.

B-6. 저장이 last-writer-wins

근거: `adms.js:294-304` `updateWiki.do` `payload = {wikiId, authorEmail, authorId, contents}` — 버전·ETag 없음. `adms.js:312` 저장자만 `closeRoom()`, 나머지 참여자에게는 "저장됨" 통지가 없음

사실|가설: 사실

A가 저장하고 방을 나간 뒤에도 B는 계속 편집하고, B가 저장하면 A의 결과를 통째로 덮는다. A-1/B-1로 이미 발산한 두 DOM 중 나중에 저장 버튼을 누른 쪽만 남는다.

B-7. 편집 진입 시 서버 문서 상태를 null로 덮음

근거: `adms.js:343` `sessionManager.setRoom(getWikiId())` — content 인자 미전달 → `session-manager.js:80` `content: undefined` → JSON에서 필드 누락 → `SessionController.java:98` `otService.setDocumentContent(sid, did, null)`

사실|가설: 사실

현재 diff 경로는 이 값을 읽지 않아 증상은 없지만, `broadcastFullDocumentState (EditorController.java:211 정의)`가 이 null을 계속 내보내고 있다. 실제 호출부는 `EditorController.java:104 (join)·:142 (leave)·WebSocketEventListener.java:120`이며, selection 경로의 호출 (`:198`)은 주석 처리되어 있다. 서버 권위 상태를 도입하려는 어떤 수정도 여기서 먼저 막힌다.

개선안

단기 완화 (구조 변경 없음)

#	조치	근거 위치	효과	영향범위	난이도
S1	<code>applyBlockDiff</code> 직후 <code>lastBlocks = getEditorBlocks(editor)</code> 재동기화	<code>adms.js:235</code> 아래 1줄	원인 A-1 직접 제거. 중복 삽입 대부분 소멸	<code>adms.js</code> 1줄	하
S2	<code>lastBlocks</code> 전진을 publish 성공 이후로 이동 + <code>_handleConnectionFailure</code> 에서 큐 비우지 말고 유지	<code>adms.js:164</code> , <code>session-manager.js:63</code>	B-2 유실 방지	<code>adms.js-session-manager.js</code>	하
S3	<code>remoteUserUpdate</code> 가 세션 없음/미연결로 전송 못 하면 false 반환, 호출부에서 <code>lastBlocks</code> 전진 취소	<code>session-manager.js:192</code>	B-2 보완	소	하
S4	<code>applyBlockDiff</code> 진입 시 로컬 캐럿 블록 인덱스를 기억하고, 적용 후 CKEditor selection 복원	<code>collab-diff.js:109</code>	B-1 시나리오 ①의 캐럿 입력 소실 완화	<code>collab-diff.js</code>	중
S5	<code>WebSocketConfig</code> 에 <code>config.setPreservePublishOrder(true)</code>	<code>WebSocketConfig.java:18</code>	B-3 완화	서버 1줄	하
S6	저장 직전 <code>data-remote-cursor</code> 노드 제거 후 <code>getData()</code>	<code>adms.js:283</code> 블록	B-5 차단	<code>adms.js</code>	하
S7	<code>sendDiffIfChanged</code> 가 [전체 delete + 전체 insert] 형태(retain 0개, 블록 N개 초과)를 만들면 전송 중단 + 재동기화 요청	<code>adms.js:160</code>	폭주 diff 서킷 브레이커	<code>adms.js</code>	하

S1~S3만 적용해도 신고된 두 증상의 다수는 사라진다. 다만 B-1 시나리오②(동시 같은 블록 편집)는 남는다 — 이건 구조 문제다.

근본 해결

#	조치	영향범위	난이도
R1	ops에 baseRev 부여 + 서버 권위 revision. <code>EditorController.handleSelectionUpdate</code> 를 릴레이가 아니라 "현재 rev와 baseRev가 같을 때만 수락, rev+1 후 broadcast, 불일치면 발신자에게 reject + 최신 스냅샷 push" 로 변경. 클라이언트는 reject 시 <code>setData</code> 후 <code>lastBlocks</code> 재동기화	서버 컨트롤러 + 클라 수신부	중
R2	블록에 안정 ID 부여 (<code>data-block-id</code> , 생성 시 1회 발급). diff를 <code>outerHTML</code> 위치 기반이 아니라 ID 기반으로 → A-2-A-4 소멸, 순서 역전 내성 확보	collab-diff.js 전면 + CKEditor 삽입 후	중 ~ 상
R3	블록 내부 텍스트 병합. <code>replace</code> 를 통째 교체가 아니라 블록 내 문자 단위 diff로 → B-1 시나리오①·② 해결	collab-diff.js 신규 로직	상
R4	저장 시 낙관적 락. <code>updateWiki.do</code> 에 <code>baseVersion</code> 추가, 서버가 충돌 시 409 반환	프론트 + Backend-Core	중
R5	이미 있는 OT 경로로 이관. <code>OtService</code> / <code>OtController</code> 가 revision-transform을 이미 갖고 있다. 자체 diff 프로토콜을 유지·보수하는 것보다 이쪽에 CKEditor 어댑터를 붙이는 편이 총 비용이 낮을 수 있다 (단 <code>setRoom</code> content 누락 B-7 선행 수정 필요)	프론트 동기화 계층 교체	상

권장 순서: S1 → S2/S3 → S5/S6 → (R1 + R2) → R4 → R3. R1+R2까지 가면 "수렴은 보장, 같은 문단 동시 편집은 나중 것 우선"이라는 **설명 가능한 semantics**에 도달한다. R3/R5는 그 다음 판단.

Issues/Caveats

- 서버 echo 정책은 확정됨.** `EditorController.java:191` 은 발신자 제외 없이 broadcast → **echo 있음**. 다만 `session-manager.js:165` 에서 클라이언트가 필터하므로 실질 영향 없음. brief가 요구한 "echo 유무 2분기"는 불필요해졌다.
- 정적 분석만 수행했다.** 브라우저 실행·재현 검증은 하지 않았다. 재현 시나리오는 코드 경로를 따라 구성한 것이며, 특히 B-3(순서 역전)·B-4(sanitize 드롭)·B-5(커서 저장 혼입)는 **실측 확인이 필요하다**.
- `setPreservePublishOrder` 기본값 **false**는 Spring 문서 근거이며 이 repo에 명시 설정이 없다는 것만 코드로 확인했다.
- 다중 인스턴스 배포 시 추가 결함 가능.** `WebSocketConfig.java:18` 은 `enableSimpleBroker` — 인메모리 브로커다. Broker-Hub를 2대 이상 띄우면 서로 다른 인스턴스에 붙은 사용자는 **아예 서로를 못 본다**(참여자 목록은 Redis라 보이는데 편집은 안 오는 형태). 배포 형상을 확인해야 한다. `SessionController.java:23` `activeSessions` 도 static in-memory + TODO 주석이 달려 있다.
- B-5는 미검증 가설.** CKEditor4의 `getData()` 가 `allowedContent:true` 에서 커서 span을 그대로 출력하는지 실제 확인이 필요하다.

6. `adms.js:248` 잠재 버그(범위 외, 참고). `parsed.type` 이 `diff / cursor` 가 아니면 `return` 없이 흘러 `setData(data)` 에 **JSON 문자열 원문**이 들어간다. 현재 타입이 2개뿐이라 미발현이지만, 타입을 추가하는 순간 문서가 JSON 텍스트로 덮인다. 이번 증상의 원인은 아니다.
 7. `_ensureConnected` 와 `_setupReconnectHandler` 의 `onConnect` 이중 래핑(`session-manager.js:104` vs `:45`) 을 검토했다. `_subscribeContentsChange` 가 앞서 `_unsubscribeContentsChange()` 를 호출하므로 **중복 구독은 발생하지 않는다**(중복 수신은 원인 아님). 다만 자동 재연결 시 죽은 subscription의 `unsubscribe()` 가 예외를 던지면 `:54` catch에 걸려 **구독 복구 전체가 중단**되고, 그 사용자는 계속 송신하면서 수신만 끊긴 채 발산한다 — 가설, 실측 필요.
-